

Day 8 - Notes

LWC Basics & Secure Development Notes

Goal of Day 8

Understand:

- What Lightning Web Components (LWC) are
 - Why Salesforce uses component-based UI
 - Frontend vs Backend separation
 - Reusable UI architecture
 - Basics of secure Salesforce development
-

Lightning Web Components (LWC)

What is LWC?

Lightning Web Components (LWC) is Salesforce's modern UI framework used to build fast and reusable user interfaces.

LWC is based on:

- HTML
- JavaScript
- CSS
- Web Standards

It is lightweight, faster, and more modern than Aura Components.

Why Salesforce Uses LWC

Salesforce moved toward LWC because:

- Better performance
- Faster rendering

- Reusable components
- Cleaner architecture
- Modern JavaScript support
- Easier maintenance
- Better scalability

LWC follows modern web development standards.

Component-Based UI

What is a Component?

A component is a reusable piece of UI.

Examples:

- Header
- Navigation bar
- Student card
- Attendance widget
- Notification panel

Large applications are built by combining many small components.

Benefits of Reusable Components

Reusable components help developers:

- Reduce duplicate code
 - Improve maintainability
 - Build faster
 - Keep UI consistent
 - Scale enterprise systems easily
 - Simplify debugging
-

LWC Component Structure

Each LWC mainly contains:

File	Purpose
HTML	UI structure
JavaScript	Logic and behavior
Meta XML	Component configuration

Basic LWC Example

HTML

```
<template>
  <h1>Hello Student</h1>
</template>
```

JavaScript

```
import { LightningElement } from 'lwc';

export default class HelloStudent extends LightningElement {}
```

Meta XML

Defines where the component can be used.

Aura vs LWC

Aura	LWC
Older framework	Modern framework
More complex	Simpler

Aura	LWC
Slower	Faster
Proprietary model	Web standards
Heavy framework	Lightweight

Events in LWC

LWC uses standard DOM events.

Event Communication

Parent-child communication happens using:

- CustomEvent
- dispatchEvent()
- Event handlers

Example:

```
const event = new CustomEvent('selected');  
this.dispatchEvent(event);
```

Frontend vs Backend Thinking

Frontend (UI Layer)

Handles:

- Buttons
- Forms
- Displaying data
- User interaction
- Navigation
- Input collection

Examples:

- Button click
 - Showing notifications
 - Displaying attendance
 - Opening dashboard
-

Backend (Apex Layer)

Handles:

- Business logic
- Database operations
- Validation
- Security
- Calculations
- Automation

Examples:

- Fee calculation
 - Record insertion
 - Validation rules
 - Trigger execution
-

College Management System – UI Screens

Possible UI Screens:

1. Student Registration Form
 2. Course Dashboard
 3. Attendance View
 4. Faculty Panel
 5. Notifications Widget
-

Component Thinking Example

Student Dashboard Components

Header Component

Displays:

- Logo
- Navigation
- Student name

Student Info Component

Displays:

- Student details
- Department
- Semester

Attendance Component

Displays:

- Attendance percentage
- Subject attendance

Notification Component

Displays:

- Alerts
- Announcements
- Updates

Course Component

Displays:

- Registered courses
- Credits
- Faculty info

Why Reusable Components Matter

Reusable components:

- Save development time
- Reduce repeated code
- Improve scalability
- Make updates easier
- Keep UI modular
- Improve team collaboration

Apex Security Basics

Why Security Matters

Enterprise systems store sensitive data like:

- Student records
- Fees
- Attendance
- Faculty information

Poor security can cause:

- Data leaks
- Unauthorized access
- Incorrect updates
- System compromise

Sharing Rules in Apex

with sharing

Enforces current user sharing rules.

```
public with sharing class StudentController
```

without sharing

Ignores sharing rules.

```
public without sharing class AdminController
```

inherited sharing

Uses caller's sharing mode.

```
public inherited sharing class MyClass
```

CRUD & FLS Security

Developers must check:

- Create access
- Read access
- Update access
- Delete access
- Field permissions

Methods:

- isAccessible()
- isCreateable()
- isUpdateable()
- isDeleteable()

User Mode Operations

User mode enforces:

- Sharing rules
- CRUD
- FLS

Example:

```
SELECT Id FROM Account WITH USER_MODE
```

SOQL Injection

What is SOQL Injection?

Attackers manipulate SOQL queries using unsafe user input.

Prevention Techniques

1. Bind Variables

Best solution.

```
[SELECT Id FROM Contact WHERE FirstName = :name]
```

2. escapeSingleQuotes()

```
String.escapeSingleQuotes(userInput)
```

3. Typecasting

Convert input safely.

```
Integer age
```

4. Allowlisting

Allow only approved values.

CSRF (Cross-Site Request Forgery)

What is CSRF?

A malicious website tricks a logged-in user into performing unwanted actions.

Prevention

- Use CSRF tokens
 - Validate origin headers
 - Avoid state-changing GET requests
 - Use POST/PUT requests
 - Trigger DML only after user interaction
-

SSRF (Server Side Request Forgery)

What is SSRF?

Attackers trick the server into making unauthorized internal/external requests.

Prevention

- Validate URLs
 - Use allowlisting
 - Disable unsafe redirects
 - Sanitize user input
 - Restrict network access
-

Modern Enterprise UI Architecture

Modern systems use component-based architecture because:

- Easier maintenance
 - Better scalability
 - Faster development
 - Team collaboration
 - Reusability
 - Cleaner separation of concerns
-

Separation of UI and Business Logic

UI should handle:

- Display
- User interaction

Backend should handle:

- Database
- Security
- Validation
- Calculations

Benefits:

- Better security
 - Easier maintenance
 - Cleaner architecture
 - Better scalability
-

Key Learnings

- LWC is Salesforce's modern UI framework
- LWC uses HTML, JavaScript, and XML
- Salesforce moved from Aura to LWC for better performance
- Components are reusable UI blocks
- Reusable architecture improves scalability

- Frontend handles UI interactions
- Backend handles business logic and security
- Apex security is critical in enterprise apps
- Sharing rules protect sensitive data
- CRUD and FLS checks are important
- SOQL injection must be prevented
- CSRF attacks use unwanted authenticated requests
- SSRF attacks exploit server-side requests
- Enterprise systems require modular architecture
- Modern development focuses on reusable, secure, scalable systems